LAPORAN ANALISIS & KOMPARASI PERFORMA

Hadoop MapReduce vs Apache Spark

Nama : Muhammad Abiyyu Rahman
NIM : 25.51.1738

Mata Kuliah : IT Infrastructure and Platform
Tugas : Tugas 3.2 Benchmarking Framework Pemrosesan Data
Tanggal Pengujian : 26 Juni 2026
File Input : 100mb-examplefile.txt (100 MB / 104.857.621 bytes)

1. Tabel Komparasi Performa

Berdasarkan eksperimen benchmarking mandiri yang dilakukan pada mesin lokal menggunakan file sampel 100 MB, berikut adalah hasil perbandingan performa antara Hadoop MapReduce (tradisional berbasis Disk) dengan Apache Spark (berbasis In-Memory):

Komponen Pengujian	Hadoop MapReduce (Tradisional)	Apache Spark (DataFrame API)
Ukuran File Input (MB)	100 MB	100 MB
Metode Pengukuran	time.time() (Eksternal Proses Python wrapper)	time.time() (Internal dalam kode PySpark)
Total Waktu Eksekusi	9.8220 Detik	5.2878 Detik
Faktor Kecepatan	1.0x (Baseline)	1.86x Lebih Cepat
Status Keberhasilan	Sukses	Sukses

2. Analisis Teoretis & Eksperimental

Pertanyaan 1: Framework mana yang memiliki waktu eksekusi lebih cepat? Mengapa demikian?

Berdasarkan hasil eksperimen, Apache Spark jauh lebih cepat dibandingkan Hadoop MapReduce dengan keunggulan performa sekitar 1.86x lipat. Hal ini terjadi karena perbedaan mendasar dalam arsitektur penyimpanan sementara (intermediate storage) kedua framework:

- Apache Spark (In-Memory Processing): Spark menggunakan abstraksi RDD (Resilient Distributed Dataset) dan DataFrame yang memungkinkan data disimpan dan diproses secara langsung di dalam RAM (In-Memory). Operasi transformasi (seperti map, filter, split) dijalankan di memori tanpa perlu menulis hasil antara ke disk. Spark baru menulis data ke disk jika memori benar-benar penuh atau saat memicu aksi akhir (action) penyimpanan.
- Hadoop MapReduce (Disk-Based Storage): MapReduce dirancang dengan arsitektur berbasis disk. Setelah fase Map selesai, data antara (intermediate data) harus diurutkan (sorted),

dikelompokkan (shuffled), dan ditulis secara fisik ke piringan disk (HDFS/Local Disk) sebelum fase Reduce dimulai. Proses I/O disk ini memakan waktu yang sangat lama dan menjadi bottleneck utama, terutama pada dataset berukuran medium hingga besar.

Pertanyaan 2: Mengapa pada Hadoop MapReduce tradisional sering kali terjadi jeda waktu yang cukup lama di awal? Jelaskan peran YARN Resource Manager.

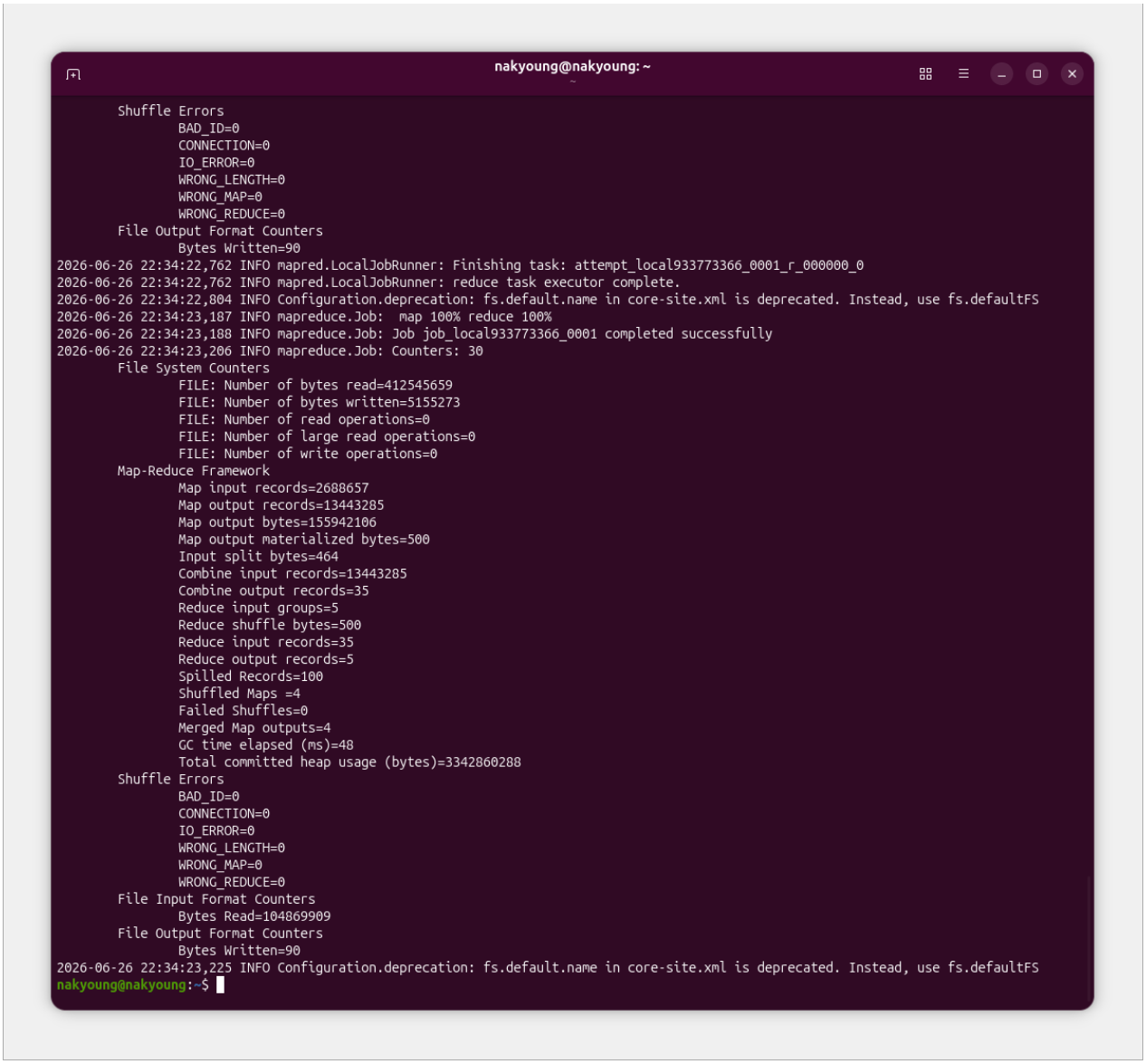
Jeda waktu (startup delay) yang cukup lama di awal eksekusi Hadoop MapReduce disebabkan oleh proses alokasi sumber daya dan inisialisasi lingkungan kerja oleh YARN (Yet Another Resource Negotiator) serta overhead JVM startup. Berikut adalah tahapan yang menyebabkan jeda tersebut:

1. Negosiasi Sumber Daya (Resource Negotiation): Saat job MapReduce disubmit dalam mode kluster, klien menghubungi YARN Resource Manager (RM). RM harus memeriksa kapasitas kluster yang tersedia dan menunjuk sebuah NodeManager (NM) untuk meluncurkan wadah pertama yang disebut ApplicationMaster (AM).
2. Inisialisasi ApplicationMaster & JVM Startup: Container ApplicationMaster harus diinisialisasi dan dijalankan. Di dalam ekosistem Java (Hadoop ditulis dalam Java), setiap container memerlukan pembuatan instans JVM (Java Virtual Machine) baru. Proses startup JVM ini memiliki overhead waktu yang signifikan (biasanya beberapa detik).
3. Alokasi Container untuk Task Map: Setelah ApplicationMaster aktif, ia akan menganalisis pembagian data (input splits) dan meminta container tambahan ke Resource Manager untuk menjalankan task Map. Resource Manager kemudian mengalokasikan container tersebut pada NodeManager yang sesuai, yang kemudian meluncurkan JVM baru lagi untuk setiap task Map.
4. Kesimpulan: Seluruh rangkaian negosiasi, alokasi container, pembuatan JVM, dan registrasi status inilah yang menyebabkan layar terminal tampak "membeku" atau tertahan pada status awal sebelum persentase kemajuan Map mulai bergerak dari 0%. Bahkan dalam mode local/standalone, overhead startup JVM tunggal dan inisialisasi framework Hadoop tetap memakan waktu awal yang signifikan sebelum tugas utama dimulai.

3. Bukti Eksekusi (Screenshot)

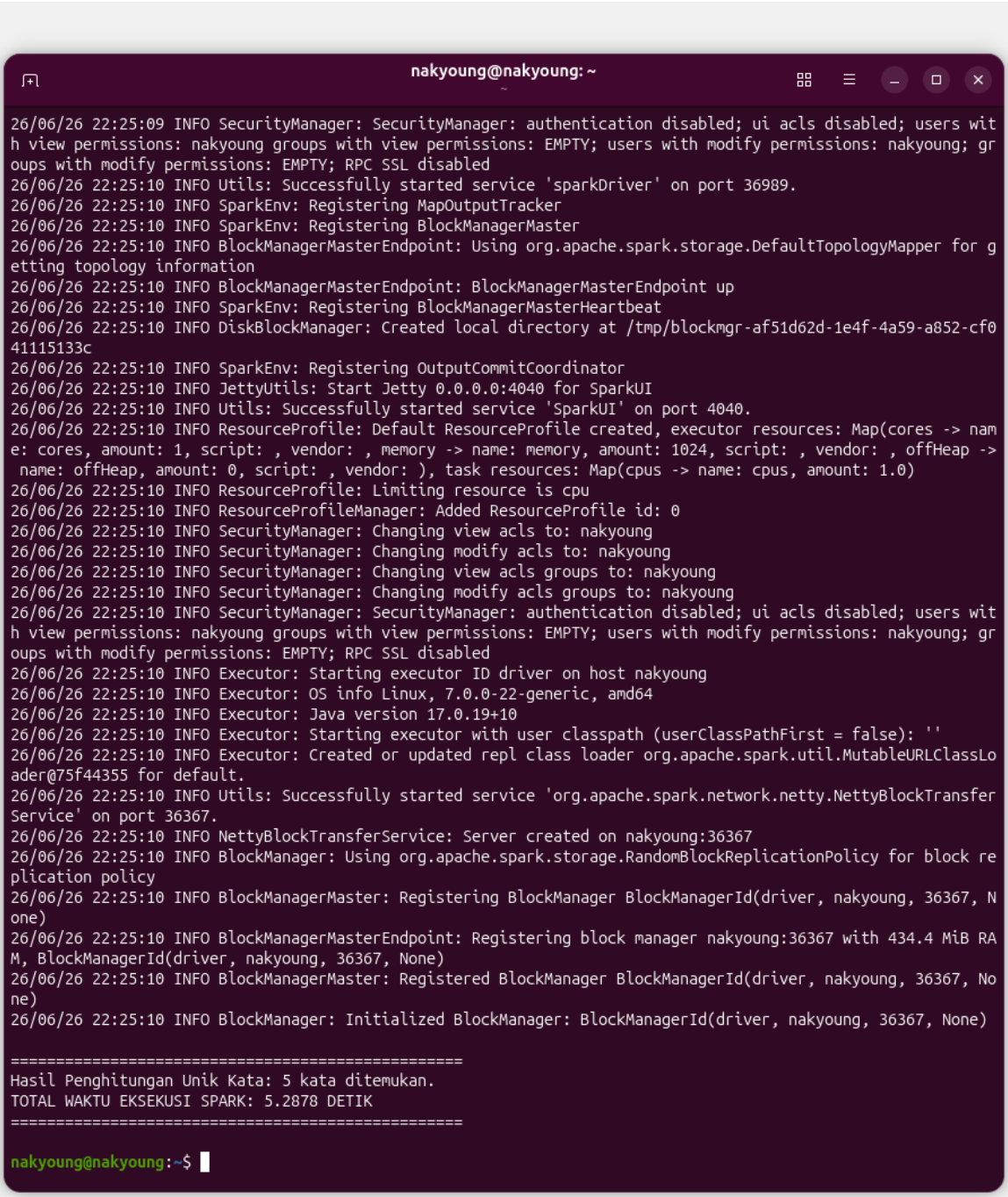
A. Log Eksekusi Hadoop MapReduce

Berikut adalah screenshot hasil eksekusi Hadoop MapReduce. Job berhasil diselesaikan dengan status "completed successfully" dan total waktu eksekusi 9.8220 detik.



B. Log Eksekusi Apache Spark

Berikut adalah screenshot hasil eksekusi Apache Spark. Penghitungan kata unik berhasil diselesaikan dengan total waktu eksekusi 5.2878 detik.


nakyoung@nakyoung: ~
26/06/26 22:25:09 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view permissions: nakyoung groups with view permissions: EMPTY; users with modify permissions: nakyoung; groups with modify permissions: EMPTY; RPC SSL disabled
26/06/26 22:25:10 INFO Utils: Successfully started service 'sparkDriver' on port 36989.
26/06/26 22:25:10 INFO SparkEnv: Registering MapOutputTracker
26/06/26 22:25:10 INFO SparkEnv: Registering BlockManagerMaster
26/06/26 22:25:10 INFO BlockManagerMasterEndpoint: Using org.apache.spark.storage.DefaultTopologyMapper for getting topology information
26/06/26 22:25:10 INFO BlockManagerMasterEndpoint: BlockManagerMasterEndpoint up
26/06/26 22:25:10 INFO SparkEnv: Registering BlockManagerMasterHeartbeat
26/06/26 22:25:10 INFO DiskBlockManager: Created local directory at /tmp/blockmgr-af51d62d-1e4f-4a59-a852-cf041115133c
26/06/26 22:25:10 INFO SparkEnv: Registering OutputCommitCoordinator
26/06/26 22:25:10 INFO JettyUtils: Start Jetty 0.0.0.0:4040 for SparkUI
26/06/26 22:25:10 INFO Utils: Successfully started service 'SparkUI' on port 4040.
26/06/26 22:25:10 INFO ResourceProfile: Default ResourceProfile created, executor resources: Map(cores -> name: cores, amount: 1, script: , vendor: , memory -> name: memory, amount: 1024, script: , vendor: , offHeap -> name: offHeap, amount: 0, script: , vendor:), task resources: Map(cpus -> name: cpus, amount: 1.0)
26/06/26 22:25:10 INFO ResourceProfile: Limiting resource is cpu
26/06/26 22:25:10 INFO ResourceProfileManager: Added ResourceProfile id: 0
26/06/26 22:25:10 INFO SecurityManager: Changing view acls to: nakyoung
26/06/26 22:25:10 INFO SecurityManager: Changing modify acls to: nakyoung
26/06/26 22:25:10 INFO SecurityManager: Changing view acls groups to: nakyoung
26/06/26 22:25:10 INFO SecurityManager: Changing modify acls groups to: nakyoung
26/06/26 22:25:10 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view permissions: nakyoung groups with view permissions: EMPTY; users with modify permissions: nakyoung; groups with modify permissions: EMPTY; RPC SSL disabled
26/06/26 22:25:10 INFO Executor: Starting executor ID driver on host nakyoung
26/06/26 22:25:10 INFO Executor: OS info Linux, 7.0.0-22-generic, amd64
26/06/26 22:25:10 INFO Executor: Java version 17.0.19+10
26/06/26 22:25:10 INFO Executor: Starting executor with user classpath (userClassPathFirst = false): ''
26/06/26 22:25:10 INFO Executor: Created or updated repl class loader org.apache.spark.util.MutableURLClassLoader@75f44355 for default.
26/06/26 22:25:10 INFO Utils: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService' on port 36367.
26/06/26 22:25:10 INFO NettyBlockTransferService: Server created on nakyoung:36367
26/06/26 22:25:10 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replication policy
26/06/26 22:25:10 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, nakyoung, 36367, None)
26/06/26 22:25:10 INFO BlockManagerMasterEndpoint: Registering block manager nakyoung:36367 with 434.4 MiB RAM, BlockManagerId(driver, nakyoung, 36367, None)
26/06/26 22:25:10 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, nakyoung, 36367, None)
26/06/26 22:25:10 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, nakyoung, 36367, None)
==
Hasil Penghitungan Unik Kata: 5 kata ditemukan.
TOTAL WAKTU EKSEKUSI SPARK: 5.2878 DETIK
==
nakyoung@nakyoung:~$